SAVANNA SENTINEL

Sigill

Deployment Requirements

Last Revision: 9 July 2026

David van Rooijen

Dandré Nel

Daniel Cohen

Stephan Kritzinger

Micheal Koch

TABLE OF CONTENTS

Deployment

Savanna Sentinel runs on a single AWS EC2 instance, with the full stack under Docker Compose. This covers how it's deployed, what's needed to stand up a new instance, how the CD pipeline works, and how to roll back.

Environments

Environment	Where it runs	URL
Development	Local machine, `docker compose up`	http://localhost:5173
Production	AWS EC2, `docker compose --profile prod up`	see README

No separate staging environment exists. Pull requests with required CI checks (`backend-ci.yml`, `frontend-ci.yml`) are the gate before anything reaches `main`, and only `main` auto-deploys.

Infrastructure

- One EC2 instance (Ubuntu 24.04, t3.small) with an Elastic IP.
- Services: `db` (Postgres+PostGIS), `redis`, `minio`, `backend`, `frontend`, `caddy`.
- `caddy` is the only service exposed on ports 80/443 - everything else stays on the internal Compose network.
- TLS comes from Let's Encrypt via Caddy, for whatever hostname `SITE_ADDRESS` is set to. We currently use a free sslip.io hostname (`<elastic-ip>.sslip.io`) instead of a purchased domain - it resolves to the instance's IP automatically, and Caddy can still get a real certificate for it.

First-time setup

1.AWS

- Create an EC2 key pair (download the `.pem`).
- Create a security group allowing inbound `22`, `80`, `443`.

2.Launch

Ubuntu Server 24.04 LTS, `t3.small`, 30GB gp3 storage, using the key pair and security group above. Allocate an Elastic IP and associate it with the instance.

3.Install Docker

```
ssh -i your-key.pem ubuntu@<elastic-ip>
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
# log out and back in for the group change to apply
```

4.Clone and Configure

```
git clone https://github.com/COS301-SE-2026/Savanna-Sentinel.git
cd Savanna-Sentinel
```

Generate a JWT Secret:

```
openssl rand -hex 32
```

Root `.env` from `.env.example:`

```
cp .env.example .env
nano .env
```

```
POSTGRES_USER=sentinel
POSTGRES_PASSWORD=<strong password>
POSTGRES_DB=savanna_sentinel
MINIO_ROOT_USER=sentinel_minio
MINIO_ROOT_PASSWORD=<strong password>
SITE_ADDRESS=<elastic-ip>.sslip.io
```

`backend/.env` from `backend/.env.example`:

```
cp backend/.env.example backend/.env
nano backend/.env
```

```
DATABASE_URL=postgresql+asyncpg://sentinel:<same
POSTGRES_PASSWORD>@db/savanna_sentinel
JWT_SECRET=<paste the openssl output>
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_SECONDS=3600
REFRESH_TOKEN_EXPIRE_DAYS=7
REDIS_URL=redis://redis:6379/0
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=sentinel_minio
MINIO_SECRET_KEY=<same MINIO_ROOT_PASSWORD>
FRONTEND_ORIGIN=https://<elastic-ip>.sslip.io
```

5.First Launch

```
docker compose -f docker-compose.yml --profile prod up -d --build
```

Verify: `curl https://<elastic-ip>.sslip.io/v1/health` should return `{"status":"ok"}`.

CD Pipeline

`github/workflows/deploy.yml` runs on every push to `main`:

1. SSH into the EC2 instance using repo secrets.
2. `git fetch` and `git checkout` the commit that triggered the deploy.
3. `docker compose -f docker-compose.yml --profile prod up -d --build`.
4. `docker image prune -f`.

Secrets

Set under repo Settings -> Secrets and variables -> Actions:

Secret	Value
`EC2_HOST`	Elastic IP
`EC2_USER`	`ubuntu`
`EC2_SSH_KEY`	Private key contents (`.pem` file) for the instance's key pair

Rollback

1. GitHub -> Actions -> "Deploy to production" -> Run workflow.
2. Set `ref` to the commit SHA (or tag) to redeploy.
3. Run it - the server checks out that commit and rebuilds.

This is a plain redeploy-previous-commit.

Changelog

V1

- Initial Version